SkillBond: Reputation-Backed Micro-Escrow for Freelance Projects

Technical Documentation for Team Review and Approval

(Internal Use Only)

---

1. Document Control

Item	Details
Project Name	SkillBond
Track	Blockchain and Fintech
Document Purpose	Technical specification and feasibility assessment for team approval
Review Cycle	Draft -> Team Review -> Final Approval -> Implementation
Last Updated	[Date]

---

2. Executive Summary

SkillBond is a decentralized escrow protocol that leverages verifiable reputation to reduce friction and cost in freelance transactions. The core innovation is treating reputation as a form of collateral: freelancers with strong, cryptographically verified completion histories qualify for lower escrow stakes and reduced platform fees. Upon successful project delivery, both parties receive portable, verifiable credentials that build a reputation graph independent of any single platform.

The project addresses a documented market need: 58% of freelancers face non-payment issues, and the global freelance platforms market is projected to reach $13.8 billion by 2030. By combining smart contract escrow logic with W3C Verifiable Credentials standards, SkillBond creates a trust layer that is portable, privacy-preserving, and economically meaningful.

This document provides the technical specification, implementation plan, and risk assessment required for team approval to proceed with a 24-hour hackathon build.

---

3. Problem Statement

3.1 Market Context

The freelance economy has grown substantially, with an estimated 76.4 million freelancers in the United States alone as of 2024. Collectively, freelancers generated $1.5 trillion in earnings in 2024. Despite this scale, trust remains a persistent friction point.

3.2 Core Problems

For Freelancers:

- Payment insecurity: 58% report experiencing non-payment or delayed payment beyond 30 days

flexable.work

- Platform lock-in: Reputation built on one platform (Upwork, Fiverr) does not transfer to others
- High friction escrow: Traditional escrow services charge 5-10% fees and impose 3-5 day holds
- Dispute asymmetry: Freelancers often bear the burden of proof in payment disputes

medium.com

For Clients:

- Quality uncertainty: Difficulty verifying freelancer claims without platform-mediated reviews
- Scope creep risk: Lack of clear, enforceable delivery criteria
- Limited recourse: Dispute resolution is often slow and platform-dependent

Systemic Issues:

- Reputation is siloed: A freelancer's 5-star rating on Platform A has no weight on Platform B
- Trust is binary: Platforms typically offer only "verified" or "not verified" without nuance
- Privacy trade-offs: Building portable reputation often requires exposing sensitive transaction history

3.3 Opportunity

A protocol that makes reputation portable, verifiable, and economically valuable would:

- Reduce transaction friction for high-quality freelancers
- Lower barriers to entry for new clients seeking trusted talent
- Create positive feedback loops that reward consistent delivery
- Enable cross-platform talent mobility

---

4. Solution Overview

4.1 Core Concept

SkillBond introduces a reputation-weighted escrow mechanism:

1. **Reputation-Backed Staking**: Freelancers with verified completion credentials can lock reduced collateral or pay lower fees to enter escrow agreements
2. **Verifiable Completion Credentials**: Upon successful project delivery, both parties receive cryptographically signed credentials that build portable reputation
3. **Transparent Dispute Flow**: Time-bound, rule-based dispute resolution with clear escalation paths
4. **Cross-Platform Portability**: Credentials follow W3C Verifiable Credentials standards, enabling verification by any compliant system

4.2 Value Proposition

Stakeholder	Primary Benefit	Secondary Benefit

Freelancer	Lower escrow costs, faster payment release	Portable reputation, reduced platform dependency
Client	Higher confidence in freelancer quality	Transparent dispute process, reduced due diligence
Platform	Lower support burden, higher user retention	Interoperability with other reputation-aware systems
Ecosystem	More efficient talent allocation	Standardized reputation primitives for future innovation

4.3 Key Differentiators

- **Reputation as Collateral**: Unlike traditional escrow, SkillBond reduces friction for proven performers
- **Standards-Based Portability**: Uses W3C VC standards

www.w3.org

, not proprietary reputation scores

- **Privacy by Design**: Credentials can be selectively disclosed; raw transaction data remains private
- **Incentive Alignment**: Both parties benefit from successful completion and honest feedback

---

5. Technical Architecture

5.1 System Components

```
+----------------+     +----------------+     +----------------+
|   Frontend     |     |   Backend      |     |   Blockchain   |
|   (React/Next) |<--->|   (Node.js)    |<--->|   (Solidity)   |
+----------------+     +----------------+     +----------------+
        |                      |                      |
        v                      v                      v
+----------------+     +----------------+     +----------------+
| Wallet Connect |     | API Gateway    |     | Smart Contracts|
| (MetaMask)     |     | (REST/GraphQL) |     | - Escrow       |
+----------------+     +----------------+     | - Reputation   |
                                              | - Credentials  |
                                              +----------------+
```

5.2 Component Specifications

Frontend (React/Next.js)

- Wallet connection via WalletConnect or MetaMask SDK
- Project creation wizard with form validation
- Escrow status dashboard with real-time updates via WebSocket or polling
- Credential viewer with verification interface
- Responsive design for desktop and mobile demo

Backend (Node.js/Express)

- REST API for off-chain data (user profiles, project metadata)
- Event listener for blockchain contract events (using ethers.js web3-provider)
- Reputation scoring service: calculates weighted score from credential history
- Mock oracle service for dispute resolution (simulated for hackathon)
- IPFS or simple JSON storage for credential metadata (hashes stored on-chain)

Smart Contracts (Solidity, deployed to Polygon Amoy or Base Sepolia testnet)

- EscrowContract: Manages fund locking, release conditions, timeout logic
- ReputationRegistry: Stores credential hashes and basic scoring parameters
- SoulboundToken (ERC-4973) or SignedCredential: Non-transferable completion credentials
- DisputeResolver: Simple time-bound arbitration logic (mock for MVP)

Identity Layer

- W3C Verifiable Credentials data model v2.0
- Decentralized Identifiers (DIDs) for issuer and holder (mock or using SpruceID sandbox)
- Selective disclosure via BBS+ signatures or simple field-level masking (conceptual for MVP)

5.3 Data Flow

1. User connects wallet and creates a project proposal
2. Frontend calls backend API to validate terms and fetch freelancer reputation score
3. Backend queries ReputationRegistry contract for credential hashes associated with freelancer address
4. Reputation score is calculated off-chain using weighted average of completion count, recency, and issuer credibility
5. User signs transaction to fund escrow contract with amount adjusted by reputation discount
6. Upon delivery and approval, contract emits CredentialMinted event
7. Backend listener generates Verifiable Credential JSON, stores metadata off-chain, records hash on-chain
8. Credential is displayed in user's profile and can be shared via QR code or deep link

5.4 Technology Stack Summary

Layer	Technology	Purpose
Frontend	React 18, Next.js 14, Tailwind CSS	User interface, wallet integration
State Management	Zustand or React Context	Local UI state, wallet state
Blockchain Interaction	ethers.js v6, Wagmi	Contract calls, event listening
Smart Contracts	Solidity 0.8.x, Foundry/Hardhat	Escrow logic, credential issuance

Testnet	Polygon Amoy or Base Sepolia	Low-cost, fast transactions for demo
Backend	Node.js 20, Express, TypeScript	API, event processing, reputation logic
Storage	IPFS (via Pinata) or simple JSON endpoint	Credential metadata storage
Identity	W3C VC data model, mock DID	Standards-compliant credential format
Deployment	Vercel (frontend), Render/Railway (backend)	Rapid deployment for demo

---

6. Smart Contract Specifications

6.1 EscrowContract

```
// Simplified interface for hackathon scope
interface IEscrowContract {
    struct Project {
        address freelancer;
        address client;
        uint256 amount;
        uint256 reputationDiscount; // Basis points (e.g., 2000 = 20%
discount)
        ProjectState state;
        uint256 createdAt;
        uint256 deadline;
        bytes32 deliverableHash; // IPFS hash or content identifier
    }

    enum ProjectState { Created, Funded, Delivered, Completed,
Disputed, Refunded }

    function createProject(
        address _freelancer,
        uint256 _amount,
        uint256 _reputationDiscount,
        uint256 _durationDays,
        bytes32 _deliverableHash
    ) external returns (uint256 projectId);

    function fundProject(uint256 projectId) external payable;

    function markDelivered(uint256 projectId, bytes32 _newHash)
external;

    function approveCompletion(uint256 projectId) external;

    function initiateDispute(uint256 projectId, string calldata reason)
external;
```

```
    function resolveDispute(uint256 projectId, bool favorFreelancer)
external; // Mock oracle

    function mintCompletionCredential(uint256 projectId) external; //
Emits event for off-chain VC generation
}
```

6.2 ReputationRegistry

```
interface IReputationRegistry {
    struct CredentialRecord {
        bytes32 credentialHash; // Hash of off-chain VC JSON
        address issuer; // Client or platform address
        uint256 issuedAt;
        uint256 weight; // Credibility weight of issuer (simple
heuristic)
    }

    function registerCredential(
        address freelancer,
        bytes32 credentialHash,
        address issuer,
        uint256 weight
    ) external;

    function getCredentialHashes(address freelancer) external view
returns (bytes32[] memory);

    function calculateReputationScore(address freelancer) external view
returns (uint256); // Basis points
}
```

6.3 Credential Schema (W3C VC Compatible)

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    "https://skillbond.example/contexts/completion-v1"
  ],
  "id": "https://skillbond.example/credentials/12345",
  "type": ["VerifiableCredential", "ProjectCompletionCredential"],
  "issuer": {
    "id": "did:ethr:0xClientAddress",
    "name": "Client Name (optional)"
  },
  "issuanceDate": "2024-01-15T10:30:00Z",
  "credentialSubject": {
    "id": "did:ethr:0xFreelancerAddress",
    "projectId": "12345",
    "projectType": "web-development",
    "durationDays": 14,
    "outcome": "completed-satisfactorily",
    "clientSatisfaction": 5,
    "deliverableHash": "bafybeihkoviema5eqheudqvnu6..."
  },
```

```
  "proof": {
    "type": "EcdsaSecp256k1Signature2019",
    "created": "2024-01-15T10:30:00Z",
    "verificationMethod": "did:ethr:0xClientAddress#controller",
    "proofPurpose": "assertionMethod",
    "jws": "eyJhbGciOiJFUzI1NksifQ..."
  }
}
```

Note: For hackathon MVP, the proof can be a simplified ECDSA signature over the credential hash, with full VC compliance noted as future work.

---

7. API Design and Data Models

7.1 Core Endpoints

GET /api/reputation/{address}

- Returns calculated reputation score and summary of credentials
- Response:

```
  {
    "address": "0x...",
    "reputationScore": 8500, // Basis points (85.00%)
    "credentialCount": 12,
    "averageRating": 4.8,
    "lastActivity": "2024-01-10T14:22:00Z"
  }
```

POST /api/projects

- Creates a new project proposal (off-chain metadata)
- Request body includes terms, deliverable description, timeline
- Returns projectId for on-chain transaction reference

GET /api/projects/{projectId}

- Returns project status, escrow state, and associated credential data

POST /api/credentials/{projectId}/mint

- Triggered by contract event; generates and stores VC JSON
- Returns credential URL and verification link

7.2 Data Models (TypeScript Interfaces)

```
interface Project {
  id: string; // UUID for off-chain reference
  onChainId?: number; // Smart contract project ID
  freelancer: string; // Ethereum address
  client: string;
  amount: bigint; // Wei
```

```
  reputationDiscount: number; // Basis points
  state: 'created' | 'funded' | 'delivered' | 'completed' | 'disputed';
  deliverableHash: string; // IPFS CID
  createdAt: Date;
  deadline: Date;
}

interface CompletionCredential {
  id: string; // Credential URL
  projectId: string;
  issuer: string; // DID or address
  subject: string; // Freelancer DID or address
  issuanceDate: string; // ISO 8601
  metadata: {
    projectType: string;
    durationDays: number;
    outcome: string;
    satisfactionRating: number; // 1-5
  };
  proof: {
    type: string;
    signature: string;
    verificationMethod: string;
  };
  onChainHash: string; // Bytes32 stored in ReputationRegistry
}
```

8. User Flows

8.1 Freelancer Flow

1. Connect wallet to SkillBond dApp
2. View reputation dashboard: score, credential history, active projects
3. Receive project proposal from client (or browse opportunities)
4. Review terms; system displays applicable reputation discount
5. Client funds escrow; freelancer receives notification
6. Complete work and submit deliverable (upload or link)
7. Client approves; funds release automatically
8. Completion credential minted and added to freelancer's portable profile

8.2 Client Flow

1. Connect wallet and create project proposal
2. Enter freelancer address; system fetches and displays reputation score
3. Set terms, budget, timeline, and deliverable criteria
4. Fund escrow contract (amount adjusted by freelancer's reputation discount)
5. Monitor progress; receive delivery notification

6. Review deliverable and approve completion
7. Receive completion credential for own reputation graph (optional)
8. Credential can be used to verify past engagements when hiring again

8.3 Verification Flow (Third Party)

1. Freelancer shares credential verification link or QR code
2. Verifier (new client, platform) opens link
3. System fetches credential JSON and on-chain hash
4. Cryptographic signature verified against issuer's public key
5. Credential validity and revocation status checked
6. Verifier sees attested claims without accessing raw transaction history

---

9. MVP Scope and 24-Hour Implementation Plan

9.1 In Scope for Hackathon

Core Functionality

- Wallet connection and testnet setup (Polygon Amoy)
- Project creation with fixed-price, single-milestone terms
- Reputation lookup using pre-seeded credential data
- Escrow simulation: fund, deliver, approve flow with contract-like state
- Credential generation: signed JSON with basic metadata, displayed in profile
- Simple reputation score: weighted average of completion count and rating
- Demo script with pre-funded test wallets and sample project data

Technical Deliverables

- Deployed smart contracts on testnet (EscrowContract, ReputationRegistry)
- Functional React frontend with three core user flows
- Node.js backend with reputation scoring API and event listener
- Sample Verifiable Credentials in W3C-compatible format
- Documentation: architecture diagram, API spec, demo script

9.2 Out of Scope (Post-Hackathon)

- Real fiat on-ramps or complex payment rails
- Multi-milestone, hourly, or retainer project types
- Full ZK-proof implementation for selective disclosure
- Production-grade dispute oracle with decentralized arbitration

- Cross-chain support or mainnet deployment
- Advanced anti-sybil mechanisms for reputation issuance

9.3 Hour-by-Hour Plan (24 Hours)

Hours 1-4: Foundation

- Set up development environment: Foundry/Hardhat, Next.js, Node.js
- Initialize Git repo, configure testnet wallets, fund with test tokens
- Scaffold smart contracts: EscrowContract, ReputationRegistry
- Write and test basic contract functions locally

Hours 5-10: Core Logic

- Deploy contracts to testnet; verify on block explorer
- Build backend API: reputation scoring, event listener, credential generation
- Implement frontend wallet connection and project creation flow
- Integrate contract ABIs with frontend for transaction signing

Hours 11-18: Integration and Polish

- Connect frontend to backend API and blockchain events
- Implement credential display and verification interface
- Add basic error handling, loading states, and user feedback
- Prepare sample data: pre-seeded credentials, test projects, demo accounts

Hours 19-24: Demo Preparation

- Record fallback demo video in case of live issues
- Write demo script with timed "wow moments"
- Prepare pitch slides: problem, solution, architecture, impact
- Conduct dry run; identify and mitigate last-minute risks
- Final testing: end-to-end flow with pre-funded accounts

9.4 Success Criteria for MVP

- User can connect wallet and view reputation score
- Client can create and fund a project with reputation-adjusted terms
- Freelancer can mark delivery and client can approve completion
- Completion credential is generated and displayed in freelancer profile
- Third party can verify credential authenticity via public link
- Entire flow completes in under 3 minutes for demo purposes

---

10. Testing Strategy

10.1 Smart Contract Testing

- Unit tests for each contract function using Foundry/Hardhat
- Fork mainnet state for realistic gas estimation (optional)
- Test edge cases: timeout, dispute initiation, re-entrancy protection
- Verify event emission for off-chain listeners

10.2 Integration Testing

- Mock backend services for frontend development
- Test API endpoints with sample requests/responses
- Verify event flow: contract emit -> backend listener -> frontend update
- End-to-end test with pre-funded test accounts

10.3 Demo Resilience

- Pre-record critical flow segments as backup
- Prepare static screenshots for each major UI state
- Have a "safe mode" demo path that skips blockchain interaction if network is congested
- Document fallback steps for common failure scenarios

---

11. Deployment and Demo Plan

11.1 Pre-Hackathon Preparation

- Create organization accounts: Vercel, Render, Pinata (IPFS), Polygon scan
- Pre-fund test wallets with sufficient testnet tokens
- Prepare sample credential data and project templates
- Draft pitch deck outline and demo script

11.2 Live Demo Sequence (3 Minutes)

1. **Problem (30 sec)**: "Freelancers lose $X billion to payment disputes annually..."
2. **Solution (45 sec)**: Live walkthrough of reputation-weighted escrow creation
3. **Technical Depth (45 sec)**: Show credential minting and verification process
4. **Impact (30 sec)**: Side-by-side comparison: traditional escrow vs. SkillBond
5. **Call to Action (30 sec)**: Vision for portable reputation ecosystem

11.3 Submission Package

- Project repository with README, architecture diagram, and setup instructions
- Deployed demo URLs: frontend, API, contract addresses

- Pitch deck (PDF) and demo script
- Short video walkthrough (optional but recommended)

---

12. Risk Assessment and Mitigation

Risk	Likelihood	Impact	Mitigation
Smart contract bug or testnet issue	Medium	High	Use OpenZeppelin templates; test on local fork; have mocked fallback flow
Wallet onboarding friction during demo	High	Medium	Pre-fund burner accounts; save session state; prepare screen recording backup
Reputation system perceived as arbitrary	Medium	Medium	Keep scoring transparent and simple; document formula; note advanced anti-sybil as future work
Scope creep during build	High	High	Strictly adhere to MVP scope; park "nice-to-have" features in roadmap slide
Network congestion or slow transactions	Medium	Medium	Use L2 testnet (Polygon/Base); set realistic gas limits; have static demo fallback
Team coordination challenges	Medium	Medium	Define clear roles; schedule brief syncs; use shared task board (GitHub Projects)

---

13. Team Roles and Responsibilities

Role	***Responsibilities***	***Skills Required***
Smart Contract Developer	Write, test, deploy EscrowContract and ReputationRegistry; handle event emission	Solidity, Foundry/Hardhat, Ethereum fundamentals
Frontend Engineer	Build React interface for project flow, wallet connection, credential display; integrate with contracts	React/Next.js, ethers.js/Wagmi, responsive design
Backend Engineer	Implement API for reputation scoring, event listening, credential generation; manage mock oracle	Node.js, Express, TypeScript, basic cryptography
Product/UX Designer	Design clear user flows, prepare demo script and slides, ensure cohesive narrative	Figma, user journey mapping, presentation design
Pitch Lead (shared)	Craft problem statement, competitive differentiation, impact narrative; prepare Q&A responses	Storytelling, technical communication, quick thinking

Note: Roles can be combined for smaller teams; prioritize contract + frontend coverage.

---

14. Success Metrics and Evaluation Criteria

14.1 Technical Success

- All core user flows function end-to-end in demo environment
- Smart contracts pass basic security checks (no obvious re-entrancy, overflow)
- Credentials follow W3C VC data model structure
- System handles edge cases gracefully (timeout, invalid input)

14.2 Presentation Success

- Problem statement is clear, data-backed, and relatable
- Solution demonstration is smooth, under 3 minutes, with visible "wow" moments
- Technical architecture is explained at appropriate depth for judge audience
- Differentiation from existing solutions is articulated

14.3 Impact Potential

- Addresses a documented pain point with measurable market size
- Demonstrates a novel application of blockchain primitives (reputation as collateral)
- Shows a credible path to post-hackathon development
- Aligns with hackathon track themes (Blockchain and Fintech)

Glossary

- **Basis Points**: One-hundredth of a percent (1 bp = 0.01%); used for precise fee/discount representation
- **DID (Decentralized Identifier)**: A portable, cryptographically verifiable identifier not controlled by any central authority
- **Escrow**: A financial arrangement where a third party holds funds until contractual conditions are met
- **Soulbound Token (SBT)**: A non-transferable NFT representing reputation, affiliation, or achievement (ERC-4973)
- **Verifiable Credential (VC)**: A tamper-evident digital credential following W3C standards, cryptographically signed by an issuer
- **Weighted Average**: Reputation scoring method that gives more influence to recent or high-credibility credentials